Trabalho Pratico 01 - StarFleet Protocol

Redes de Computadores — 2025/2

Aluno: João Vaz Matrícula: 2023099166

23 de outubro de 2025

1 Introdução

O trabalho *StarFleet Protocol* implementa um sistema cliente-servidor TCP em C, que simula combates por turnos entre uma nave controlada pelo jogador e uma nave inimiga controlada pelo servidor. Cada rodada envolve a escolha de ações (ataque, defesa, fuga), o cálculo do resultado conforme regras determinísticas e a atualização dos pontos de vida (HP). A simulação termina por destruição de uma das naves ou fuga.

2 Objetivos

O objetivo geral é desenvolver uma aplicação TCP completa e funcional que troque mensagens entre cliente e servidor para simular o combate estelar, incluindo:

- **Conexão:** servidor inicializado com protocolo (`v4` ou `v6`) e porta; cliente conecta via IP e porta.
- **Simulação:** cada nave inicia com 100 HP e as ações causam ou evitam dano conforme regras fixas.
- **Inventário:** ao final, são informados HP final, torpedos e escudos usados e resultado textual.

3 Arquitetura do Sistema

O sistema é composto por três arquivos principais: `server.c`, `client.c` e `Makefile`. O servidor e o cliente compartilham a estrutura `BattleMessage`, garantindo comunicação binária consistente.

3.1 Servidor (server.c)

O servidor aceita um cliente por vez. O protocolo e a porta são passados por argumento:

```
./server v4 5001   # IPv4
./server v6 5001   # IPv6
```

Após o `accept`, o servidor envia uma mensagem de boas-vindas (`MSG_INIT`) e inicia o ciclo:

1. Solicita uma ação do cliente (`MSG_ACTION_REQ`);
2. Recebe resposta (`MSG_ACTION_RES`);
3. Sorteia a ação do inimigo e aplica as regras de combate;
4. Envia o resultado (`MSG_BATTLE_RESULT`);
5. Se o jogo terminou, envia `MSG_INVENTORY`.

A função `msg_action_res()` implementa toda a lógica do turno. As ações são sorteadas em $[0, 4]$ e o cálculo do dano segue o enunciado: `Shields Up` bloqueia ataques, `Cloaking` evita torpedos, `Torpedo` vence `Laser`, e ataques idênticos causam dano mútuo. A cada turno o servidor atualiza HPs e contadores (`torpedoes`, `shields`) e envia o feedback textual.

3.2 Cliente (client.c)

O cliente conecta-se ao servidor informando endereço e porta:

```
./client 127.0.0.1 5001   # IPv4
./client ::1 5001          # IPv6
```

Após conectar, o cliente:

1. Recebe e imprime mensagens (`MSG_INIT`, `MSG_BATTLE_RESULT`);
2. Ao receber `MSG_ACTION_REQ`, lê a entrada do usuário (0–4);
3. Envia `MSG_ACTION_RES` com a ação escolhida;
4. Exibe feedback até o encerramento (`MSG_INVENTORY` ou `MSG_ESCAPE`).

A função `msg_action_req()` garante a validação da entrada, evitando valores fora de 0–4 e exibindo mensagens de erro conforme o exemplo 10 do enunciado. O cliente encerra corretamente a sessão com `MSG_GAME_OVER` após o inventário final.

3.3 Makefile

O `Makefile` automatiza a compilação, gerando os binários em `bin/`. Os alvos principais são:

```
make run_server_v4   # Executa servidor IPv4
make run_server_v6   # Executa servidor IPv6
make run_client_v4   # Executa cliente IPv4
make run_client_v6   # Executa cliente IPv6
make clean           # Remove binários
```

4 Protocolo de Comunicação

O protocolo define a estrutura `BattleMessage`, composta por:

- `type`: tipo da mensagem (init, ação, resultado, inventário, etc.);
- `client_action` e `server_action`;
- `client_hp`, `server_hp`;
- `client_torpedoes`, `client_shields`;
- `message`: texto de feedback.

Essa estrutura é enviada diretamente por socket com `send`/`recv`, evitando parsing textual e simplificando o protocolo. A troca segue uma sequência determinística, impedindo estados inconsistentes entre cliente e servidor.

5 Desafios Encontrados

- **Sincronização de mensagens:** garantir que cada envio tenha resposta correspondente, evitando travamentos em `recv()`.
- **Compatibilidade IPv4/IPv6:** parametrização correta das estruturas `sockaddr_in` e `sockaddr_in6`.
- **Implementação das regras:** traduzir corretamente todas as combinações possíveis de ações em condições lógicas simples e legíveis.
- **Mensagens textuais:** construir feedbacks coerentes e compatíveis com os exemplos oficiais.

6 Estratégias Adotadas

- Estrutura única de mensagem (`BattleMessage`) para todos os tipos.
- Código modular com funções dedicadas a cada tipo de mensagem (`msg_init`, `msg_action_req`, `msg_inventory`, etc.).
- Simplicidade: apenas um cliente por vez e controle centralizado no servidor.

7 Possíveis Melhorias

- Permitir múltiplos clientes simultâneos com `fork()` ou `select()`.
- Criar logs automáticos de cada partida.
- Implementar reconexão e tratamento de desconexões abruptas.
- Adicionar mensagens coloridas e temporizações para tornar o jogo mais imersivo.

8 Conclusão

O desenvolvimento do StarFleet Protocol proporcionou uma aplicação prática dos conceitos de comunicação via sockets TCP e da interação entre processos cliente e servidor. A partir de uma implementação simples, mas completa, foi possível compreender de forma concreta como ocorre o estabelecimento de conexões, a troca estruturada de mensagens e o controle determinístico de estados em uma aplicação de rede. Entre os principais desafios enfrentados estiveram a sincronização das mensagens trocadas, a compatibilidade entre IPv4 e IPv6 e a tradução precisa das regras de combate definidas no enunciado. As estratégias adotadas priorizaram clareza, modularidade e fidelidade ao protocolo. Apesar dos resultados satisfatórios, há espaço para aprimoramentos, como a introdução de suporte a múltiplos clientes, o registro de logs de partida e uma inteligência adversária mais elaborada. Assim, o trabalho alcançou seu objetivo de unir programação em rede e lógica de jogo, demonstrando como a comunicação entre máquinas pode ser explorada de forma criativa e educativa.